

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА
Институт кибербезопасности и цифровых технологий
Кафедра КБ-4 «Интеллектуальные системы информационной
безопасности»

Отчет по практической работе № 8
по дисциплине «Специальные разделы математики»
Тема: «Классификация образов с помощью функций правдоподобия»

Выполнил: студент группы ББМО-01-25

Мухаметшин Александр Ринатович

Проверила:
Изотова Анастасия Андреевна

Москва, 2025

Ход работы

Задание 2. Классификация с тремя классами

Условие:

Система классификации документов распознает три типа документов по двум признакам:

- x_1 — частота ключевых слов
- x_2 — средняя длина предложения

Параметры классов:

Класс A_1 (технические документы):

- $P(A_1) = 0.5$
- $m_1 = [8.0, 15.0]^T$
- $C_1 = [[4.0, 1.0], [1.0, 9.0]]$

Класс A_2 (научные статьи):

- $P(A_2) = 0.3$
- $m_2 = [12.0, 20.0]^T$
- $C_2 = [[6.0, 2.0], [2.0, 16.0]]$

Класс A_3 (художественная литература):

- $P(A_3) = 0.2$
- $m_3 = [5.0, 12.0]^T$
- $C_3 = [[3.0, 0.5], [0.5, 4.0]]$

Требуется:

1. Построить байесовский классификатор для трех классов
2. Классифицировать документы: $x^{(1)} = [10.0, 18.0]^T$, $x^{(2)} = [6.0, 13.0]^T$
3. Вычислить расстояния Махаланобиса между всеми парами классов
4. Визуализировать разделяющие границы

Решение:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

class BayesianClassifier:
    def __init__(self):
        self.classes = {}
        self.n_features = None

    def add_class(self, class_name: str, prior: float, mean:
np.ndarray, cov: np.ndarray):
```

```

        if self.n_features is None:
            self.n_features = len(mean)
        elif len(mean) != self.n_features:
            raise ValueError("Размерность признаков должна быть
одинаковой")

        self.classes[class_name] = {
            'prior': prior,
            'mean': np.array(mean),
            'cov': np.array(cov),
            'cov_inv': np.linalg.inv(cov),
            'cov_det': np.linalg.det(cov)
        }

    def discriminant_function(self, x: np.ndarray, class_name:
str) -> float:
        cls = self.classes[class_name]
        diff = x - cls['mean']
        mahalanobis_sq = diff.T @ cls['cov_inv'] @ diff
        g = np.log(cls['prior']) - 0.5 * np.log(cls['cov_det'])
- 0.5 * mahalanobis_sq
        return g

    def predict(self, x: np.ndarray):
        scores = {}
        for class_name in self.classes:
            scores[class_name] = self.discriminant_function(x,
class_name)
        predicted_class = max(scores, key=scores.get)
        return predicted_class, scores

    def mahalanobis_distance(self, class1: str, class2: str,
common_cov: np.ndarray = None) -> float:
        m1 = self.classes[class1]['mean']
        m2 = self.classes[class2]['mean']

        if common_cov is None:
            c1 = self.classes[class1]['cov']
            c2 = self.classes[class2]['cov']
            common_cov = (c1 + c2) / 2

        cov_inv = np.linalg.inv(common_cov)
        diff = m1 - m2
        r_sq = diff.T @ cov_inv @ diff
        return np.sqrt(r_sq)

```

```

classifier = BayesianClassifier()

m1 = np.array([8.0, 15.0])
C1 = np.array([[4.0, 1.0], [1.0, 9.0]])
classifier.add_class('A1', 0.5, m1, C1)

m2 = np.array([12.0, 20.0])
C2 = np.array([[6.0, 2.0], [2.0, 16.0]])
classifier.add_class('A2', 0.3, m2, C2)

m3 = np.array([5.0, 12.0])
C3 = np.array([[3.0, 0.5], [0.5, 4.0]])
classifier.add_class('A3', 0.2, m3, C3)

test_samples = [np.array([10.0, 18.0]), np.array([6.0, 13.0])]

print("Классификация документов:")
for idx, x in enumerate(test_samples, 1):
    pred_class, scores = classifier.predict(x)
    print(f"\nДокумент {idx}: x = {x}")
    print("Дискриминантные функции:")
    for name, score in scores.items():
        print(f"    g_{name}(x) = {score:.4f}")
    print(f"Решение: {pred_class}")

print("\nРасстояния Махаланобиса между классами:")
pairs = [('A1', 'A2'), ('A1', 'A3'), ('A2', 'A3')]
for c1, c2 in pairs:
    r = classifier.mahalanobis_distance(c1, c2)
    print(f"r_{c1}_{c2} = {r:.4f}")

x1 = np.linspace(0, 16, 200)
x2 = np.linspace(8, 25, 200)
X1, X2 = np.meshgrid(x1, x2)
Z = np.zeros_like(X1)
class_names = list(classifier.classes.keys())

for i in range(X1.shape[0]):
    for j in range(X1.shape[1]):
        x = np.array([X1[i, j], X2[i, j]])
        pred_class, _ = classifier.predict(x)
        Z[i, j] = class_names.index(pred_class)

plt.figure(figsize=(5, 4))
plt.contourf(X1, X2, Z, levels=[-0.5, 0.5, 1.5, 2.5],
             colors=['lightblue', 'lightgreen', 'lightcoral'],

```

```

alpha=0.5)
plt.contour(X1, X2, Z, levels=[0.5, 1.5], colors='black',
linewidths=2)

colors = ['blue', 'green', 'red']
markers = ['o', 's', '^']
for idx, (class_name, cls) in
enumerate(classifier.classes.items()):
    plt.plot(cls['mean'][0], cls['mean'][1],
marker=markers[idx],
            markersize=12, color=colors[idx], label=f'Среднее
{class_name}',
            markeredgecolor='black', markeredgewidth=2)

for idx, point in enumerate(test_samples):
    pred_class, _ = classifier.predict(point)
    color_map = {'A1': 'blue', 'A2': 'green', 'A3': 'red'}
    plt.plot(point[0], point[1], 'X', markersize=12,
            color=color_map[pred_class], markeredgecolor='black',
markeredgewidth=1.5)

plt.xlabel('x1 (частота ключевых слов)')
plt.ylabel('x2 (средняя длина предложения)')
plt.title('Классификация документов (3 класса)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

Вывод:

• Классификация документов:

Документ 1: $x = [10. \ 18.]$

Дискриминантные функции:

$$g_{A1}(x) = -3.3280$$

$$g_{A2}(x) = -3.8562$$

$$g_{A3}(x) = -10.4158$$

Решение: A1

Документ 2: $x = [6. \ 13.]$

Дискриминантные функции:

$$g_{A1}(x) = -3.0994$$

$$g_{A2}(x) = -7.2801$$

$$g_{A3}(x) = -3.0967$$

Решение: A3

Расстояния Махаланобиса между классами:

$$r_{A1_A2} = 2.0972$$

$$r_{A1_A3} = 1.8568$$

$$r_{A2_A3} = 3.8325$$



Задание 3. Классификация с равными ковариационными матрицами

Условие:

Система биометрической аутентификации использует два признака:

- x_1 — время нажатия клавиши (мс)
- x_2 — время между нажатиями (мс)

Два пользователя:

Пользователь 1:

- $P(A_1) = 0.5$
- $m_1 = [150, 100]^T$

Пользователь 2:

- $P(A_2) = 0.5$
- $m_2 = [180, 120]^T$

Общая ковариационная матрица:

- $C = [[400, 50], [50, 225]]$

Требуется:

1. Вывести уравнение разделяющей гиперплоскости
2. Построить линейный классификатор
3. Вычислить расстояние Махаланобиса
4. Оценить вероятность ошибки
5. Визуализировать линейную разделяющую границу

Решение:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import erf

class LinearBayesianClassifier:
    def __init__(self):
        self.classes = {}
        self.n_features = None
        self.common_cov = None

    def add_class(self, class_name: str, prior: float, mean:
np.ndarray):
        if self.n_features is None:
            self.n_features = len(mean)

        self.classes[class_name] = {
            'prior': prior,
            'mean': np.array(mean)
        }

    def set_common_cov(self, cov: np.ndarray):
```

```

        self.common_cov = np.array(cov)
        self.cov_inv = np.linalg.inv(cov)

    def linear_discriminant(self, x: np.ndarray, class_name: str)
-> float:
        cls = self.classes[class_name]
        m = cls['mean']
        g = m.T @ self.cov_inv @ x - 0.5 * m.T @ self.cov_inv @
m + np.log(cls['prior'])
        return g

    def predict(self, x: np.ndarray):
        scores = {}
        for class_name in self.classes:
            scores[class_name] = self.linear_discriminant(x,
class_name)
        predicted_class = max(scores, key=scores.get)
        return predicted_class, scores

    def mahalanobis_distance(self, class1: str, class2: str) ->
float:
        m1 = self.classes[class1]['mean']
        m2 = self.classes[class2]['mean']
        diff = m1 - m2
        r_sq = diff.T @ self.cov_inv @ diff
        return np.sqrt(r_sq)

    def error_probability(self, class1: str, class2: str) ->
float:
        r = self.mahalanobis_distance(class1, class2)
        p_error = 0.5 * (1 - erf(r / (2 * np.sqrt(2))))
        return p_error

    def get_hyperplane_params(self, class1: str, class2: str):
        m1 = self.classes[class1]['mean']
        m2 = self.classes[class2]['mean']

        w = self.cov_inv @ (m1 - m2)
        w0 = -0.5 * (m1.T @ self.cov_inv @ m1 - m2.T @ self.cov_inv
@ m2)

        return w, w0

classifier = LinearBayesianClassifier()

m1 = np.array([150, 100])

```



```

m2 = np.array([180, 120])
C = np.array([[400, 50], [50, 225]])

classifier.add_class('User1', 0.5, m1)
classifier.add_class('User2', 0.5, m2)
classifier.set_common_cov(C)

w, w0 = classifier.get_hyperplane_params('User1', 'User2')
print("Уравнение разделяющей гиперплоскости:")
print(f" $w^T \cdot x + w_0 = 0$ ")
print(f" $w = \{w\}$ ")
print(f" $w_0 = \{w_0:.4f\}$ ")
print(f"\nЯВНЫЙ вид:  $\{w[0]:.4f\} \cdot x_1 + \{w[1]:.4f\} \cdot x_2 + \{w_0:.4f\} = 0$ ")

r = classifier.mahalanobis_distance('User1', 'User2')
print(f"\nРасстояние Махаланобиса:  $r = \{r:.4f\}$ ")
print(f"Квадрат расстояния:  $r^2 = \{r**2:.4f\}$ ")

p_error = classifier.error_probability('User1', 'User2')
print(f"\nВероятность ошибки: P_ош =  $\{p\_error:.4f\}$  ( $\{p\_error*100:.2f\}\%$ )")

x1_range = np.linspace(120, 210, 200)
x2_range = np.linspace(70, 150, 200)
X1, X2 = np.meshgrid(x1_range, x2_range)
Z = np.zeros_like(X1)

for i in range(X1.shape[0]):
    for j in range(X1.shape[1]):
        x = np.array([X1[i, j], X2[i, j]])
        pred_class, _ = classifier.predict(x)
        Z[i, j] = 0 if pred_class == 'User1' else 1

plt.figure(figsize=(5, 4))
plt.contourf(X1, X2, Z, levels=[-0.5, 0.5, 1.5],
             colors=['lightblue', 'lightcoral'], alpha=0.5)

x1_line = np.linspace(120, 210, 100)
x2_line = -(w[0] * x1_line + w0) / w[1]
plt.plot(x1_line, x2_line, 'k-', linewidth=2, label='Разделяющая гиперплоскость')

plt.plot(m1[0], m1[1], 'bo', markersize=12, label='User1',
         markeredgewidth=2)
plt.plot(m2[0], m2[1], 'rs', markersize=12, label='User2',

```

```

        markeredgewidth=2)

plt.xlabel('x1 (время нажатия, мс)')
plt.ylabel('x2 (время между нажатиями, мс)')
plt.title('Линейная разделяющая граница')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(120, 210)
plt.ylim(70, 150)
plt.show()

```

Вывод:

Уравнение разделяющей гиперплоскости:

$$w^T * x + w_0 = 0$$

$$w = [-0.06571429 \ -0.07428571]$$

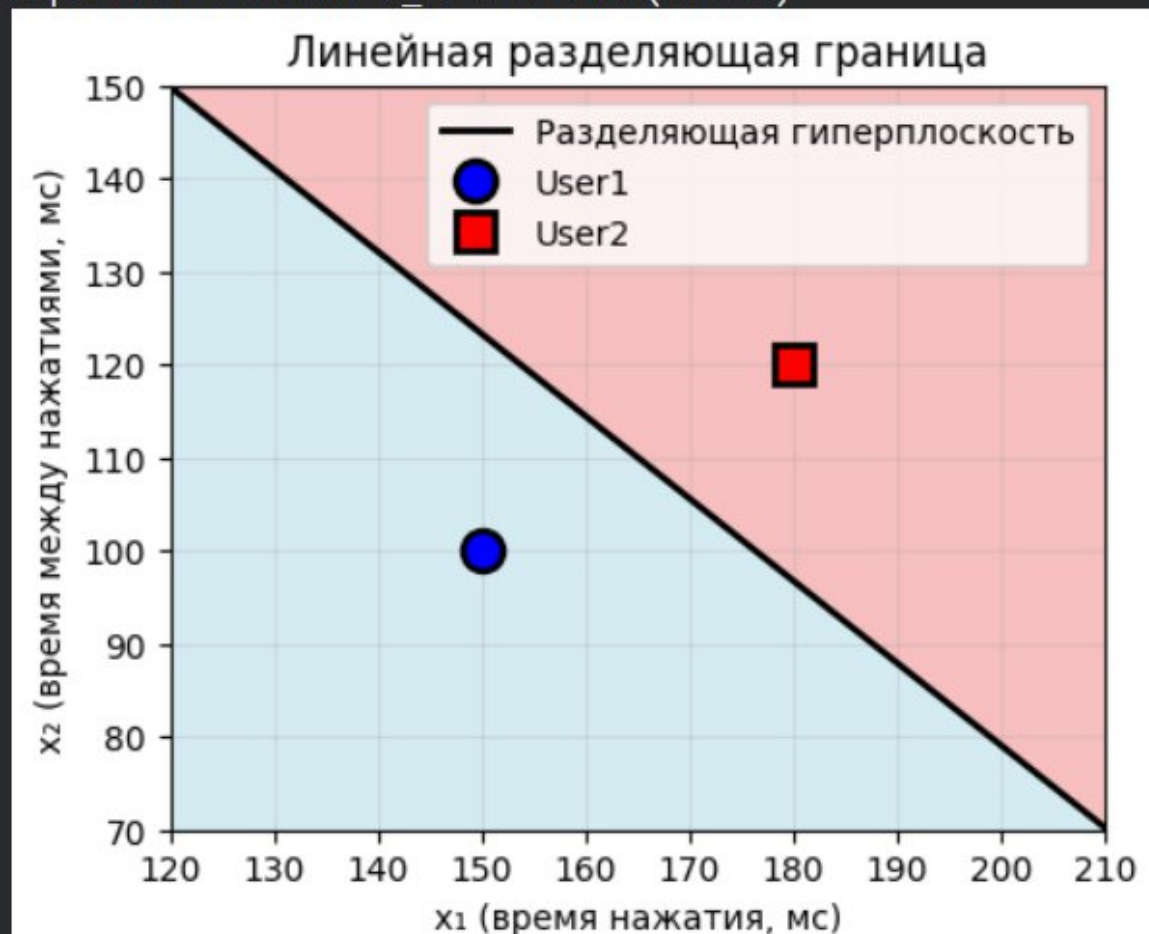
$$w_0 = 19.0143$$

$$\text{Явный вид: } -0.0657 * x_1 + -0.0743 * x_2 + 19.0143 = 0$$

Расстояние Махаланобиса: $r = 1.8593$

Квадрат расстояния: $r^2 = 3.4571$

Вероятность ошибки: $P_{\text{ош}} = 0.1763$ (17.63%)



Задание 4. Оценка параметров по обучающей выборке

Условие:

Дана обучающая выборка для двух классов:

Класс A_1 :

[[1.2, 2.3], [1.5, 2.1], [1.8, 2.5], [1.3, 2.0], [1.6, 2.4]]

Класс A_2 :

[[3.1, 1.2], [3.5, 1.5], [3.2, 1.3], [3.8, 1.1], [3.4, 1.4]]

Требуется:

1. Оценить векторы средних m_1 и m_2
2. Оценить ковариационные матрицы C_1 и C_2
3. Принять $P(A_1) = P(A_2) = 0.5$
4. Построить классификатор
5. Классифицировать новые наблюдения: $x^{(1)} = [2.0, 2.0]^T$, $x^{(2)} = [2.5, 1.8]^T$
6. Провести кросс-валидацию (leave-one-out)

Решение:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

class BayesianClassifier:
    def __init__(self):
        self.classes = {}
        self.n_features = None

    def add_class(self, class_name: str, prior: float, mean:
np.ndarray, cov: np.ndarray):
        if self.n_features is None:
            self.n_features = len(mean)

        self.classes[class_name] = {
            'prior': prior,
            'mean': np.array(mean),
            'cov': np.array(cov),
            'cov_inv': np.linalg.inv(cov),
            'cov_det': np.linalg.det(cov)
        }

    def discriminant_function(self, x: np.ndarray, class_name:
str) -> float:
        cls = self.classes[class_name]
        diff = x - cls['mean']
```

```

        mahalanobis_sq = diff.T @ cls['cov_inv'] @ diff
        g = np.log(cls['prior']) - 0.5 * np.log(cls['cov_det'])
    - 0.5 * mahalanobis_sq
    return g

    def predict(self, x: np.ndarray):
        scores = {}
        for class_name in self.classes:
            scores[class_name] = self.discriminant_function(x,
class_name)
        predicted_class = max(scores, key=scores.get)
        return predicted_class, scores

data_A1 = np.array([[1.2, 2.3], [1.5, 2.1], [1.8, 2.5], [1.3,
2.0], [1.6, 2.4]])
data_A2 = np.array([[3.1, 1.2], [3.5, 1.5], [3.2, 1.3], [3.8,
1.1], [3.4, 1.4]])

m1 = np.mean(data_A1, axis=0)
C1 = np.cov(data_A1.T)

m2 = np.mean(data_A2, axis=0)
C2 = np.cov(data_A2.T)

print("Оценка параметров по обучающей выборке:")
print(f"\nКласс A1:")
print(f"m1 = {m1}")
print(f"C1 = \n{C1}")

print(f"\nКласс A2:")
print(f"m2 = {m2}")
print(f"C2 = \n{C2}")

classifier = BayesianClassifier()
classifier.add_class('A1', 0.5, m1, C1)
classifier.add_class('A2', 0.5, m2, C2)

test_samples = [np.array([2.0, 2.0]), np.array([2.5, 1.8])]

print("\nКлассификация новых наблюдений:")
for idx, x in enumerate(test_samples, 1):
    pred_class, scores = classifier.predict(x)
    print(f"\nНаблюдение {idx}: x = {x}")
    print(f"Решение: {pred_class}")

print("\nКросс-валидация (leave-one-out):")

```

```

all_data = np.vstack([data_A1, data_A2])
all_labels = ['A1']*len(data_A1) + ['A2']*len(data_A2)

correct = 0
total = len(all_data)

for i in range(total):
    train_indices = [j for j in range(total) if j != i]

    if i < len(data_A1):
        train_A1 = data_A1[[j for j in range(len(data_A1)) if
j != i]]
        train_A2 = data_A2
    else:
        train_A1 = data_A1
        train_A2 = data_A2[[j-len(data_A1) for j in train_indices
if j >= len(data_A1)]]

    m1_cv = np.mean(train_A1, axis=0)
    C1_cv = np.cov(train_A1.T)
    m2_cv = np.mean(train_A2, axis=0)
    C2_cv = np.cov(train_A2.T)

    clf_cv = BayesianClassifier()
    clf_cv.add_class('A1', 0.5, m1_cv, C1_cv)
    clf_cv.add_class('A2', 0.5, m2_cv, C2_cv)

    pred, _ = clf_cv.predict(all_data[i])
    if pred == all_labels[i]:
        correct += 1

accuracy = correct / total
print(f"Точность: {accuracy:.2%} ({correct}/{total})")

```

Вывод:

Оценка параметров по обучающей выборке:

Класс A1:

$m_1 = [1.48 \ 2.26]$

$C_1 =$

$\begin{bmatrix} 0.057 & 0.0315 \\ 0.0315 & 0.043 \end{bmatrix}$

Класс A2:

$m_2 = [3.4 \ 1.3]$

$C_2 =$

$\begin{bmatrix} 0.075 & -0.0075 \\ -0.0075 & 0.025 \end{bmatrix}$

Классификация новых наблюдений:

Наблюдение 1: $x = [2. \ 2.]$

Решение: A1

Наблюдение 2: $x = [2.5 \ 1.8]$

Решение: A2

Кросс-валидация (leave-one-out):

Точность: 90.00% (9/10)

Задание 5. Классификация с неравными априорными вероятностями

Условие:

Система обнаружения спама классифицирует email по признакам:

- x_1 — количество восклицательных знаков
- x_2 — количество ссылок

Параметры:

Класс A_1 (легитимная почта):

- $P(A_1) = 0.95$
- $m_1 = [0.5, 1.0]^T$
- $C_1 = [[0.25, 0.1], [0.1, 0.5]]$

Класс A_2 (спам):

- $P(A_2) = 0.05$
- $m_2 = [3.0, 5.0]^T$
- $C_2 = [[1.0, 0.5], [0.5, 2.0]]$

Требуется:

1. Построить классификатор с исходными априорными вероятностями
2. Классифицировать письма: $x^{(1)} = [1.5, 2.5]^T$, $x^{(2)} = [2.5, 4.0]^T$
3. Исследовать влияние изменения $P(A_1)$ и $P(A_2)$ на классификацию

4. Построить ROC-кривую, варьируя пороговое значение

5. Найти оптимальный порог для минимизации ложных срабатываний

Решение:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

class BayesianClassifier:
    def __init__(self):
        self.classes = {}
        self.n_features = None

    def add_class(self, class_name: str, prior: float, mean:
np.ndarray, cov: np.ndarray):
        if self.n_features is None:
            self.n_features = len(mean)

        self.classes[class_name] = {
            'prior': prior,
            'mean': np.array(mean),
            'cov': np.array(cov),
            'cov_inv': np.linalg.inv(cov),
            'cov_det': np.linalg.det(cov)
        }

    def discriminant_function(self, x: np.ndarray, class_name:
str) -> float:
        cls = self.classes[class_name]
        diff = x - cls['mean']
        mahalanobis_sq = diff.T @ cls['cov_inv'] @ diff
        g = np.log(cls['prior']) - 0.5 * np.log(cls['cov_det'])
- 0.5 * mahalanobis_sq
        return g

    def predict(self, x: np.ndarray):
        scores = {}
        for class_name in self.classes:
            scores[class_name] = self.discriminant_function(x,
class_name)
        predicted_class = max(scores, key=scores.get)
        return predicted_class, scores

    def predict_proba(self, x: np.ndarray):
        likelihoods = {}
        for class_name in self.classes:
```

```

        cls = self.classes[class_name]
        rv = multivariate_normal(cls['mean'], cls['cov'])
        likelihoods[class_name] = cls['prior'] * rv.pdf(x)

    total = sum(likelihoods.values())
    probas = {k: v/total for k, v in likelihoods.items()}
    return probas

m1 = np.array([0.5, 1.0])
C1 = np.array([[0.25, 0.1], [0.1, 0.5]])

m2 = np.array([3.0, 5.0])
C2 = np.array([[1.0, 0.5], [0.5, 2.0]])

classifier = BayesianClassifier()
classifier.add_class('A1', 0.95, m1, C1)
classifier.add_class('A2', 0.05, m2, C2)

test_samples = [np.array([1.5, 2.5]), np.array([2.5, 4.0])]

print("Классификация с исходными вероятностями P(A1)=0.95, P(A2)=0.05:")
for idx, x in enumerate(test_samples, 1):
    pred_class, scores = classifier.predict(x)
    probas = classifier.predict_proba(x)
    print(f"\nПисьмо {idx}: x = {x}")
    print(f"Решение: {pred_class}")
    print(f"P(A1|x) = {probas['A1']:.4f}, P(A2|x) = {probas['A2']:.4f}")

print("\nВлияние априорных вероятностей на классификацию:")
prior_values = [(0.95, 0.05), (0.8, 0.2), (0.6, 0.4), (0.5, 0.5)]
test_point = np.array([2.0, 3.0])

print(f"\nТестовая точка: x = {test_point}")
print(f"{'P(A1)':<8} | {'P(A2)':<8} | {'Решение':<10}")
print("-" * 40)

for p1, p2 in prior_values:
    clf_temp = BayesianClassifier()
    clf_temp.add_class('A1', p1, m1, C1)
    clf_temp.add_class('A2', p2, m2, C2)
    pred, _ = clf_temp.predict(test_point)
    print(f"{p1:<8.2f} | {p2:<8.2f} | {pred:<10}")

thresholds = np.linspace(-10, 10, 100)

```



```

x1_grid = np.linspace(-1, 5, 100)
x2_grid = np.linspace(-1, 7, 100)

tpr_list = []
fpr_list = []

n_samples = 200
np.random.seed(42)
samples_A1 = np.random.multivariate_normal(m1, C1, n_samples)
samples_A2 = np.random.multivariate_normal(m2, C2, n_samples)

for threshold in thresholds:
    tp = fp = tn = fn = 0

    for sample in samples_A2:
        score_diff = classifier.discriminant_function(sample,
'A2') - \
            classifier.discriminant_function(sample, 'A1')
        if score_diff > threshold:
            tp += 1
        else:
            fn += 1

    for sample in samples_A1:
        score_diff = classifier.discriminant_function(sample,
'A2') - \
            classifier.discriminant_function(sample, 'A1')
        if score_diff > threshold:
            fp += 1
        else:
            tn += 1

    tpr = tp / (tp + fn) if (tp + fn) > 0 else 0
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
    tpr_list.append(tpr)
    fpr_list.append(fpr)

plt.figure(figsize=(5, 4))
plt.plot(fpr_list, tpr_list, 'b-', linewidth=2)
plt.plot([0, 1], [0, 1], 'r--', label='Случайный классификатор')
plt.xlabel('FPR (False Positive Rate)')
plt.ylabel('TPR (True Positive Rate)')
plt.title('ROC-кривая')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim([0, 1])

```

```
plt.ylim([0, 1])
plt.show()

min_fp_idx = np.argmin([fpr for fpr in fpr_list if fpr < 0.1] or
                        fpr_list)
optimal_threshold = thresholds[min_fp_idx]
print(f"\nОптимальный порог для минимизации ложных срабатываний:
{optimal_threshold:.4f}")
print(f"FPR      =      {fpr_list[min_fp_idx]:.4f},      TPR      =
{tpr_list[min_fp_idx]:.4f}")
```

Вывод:

Классификация с исходными вероятностями $P(A1)=0.95$, $P(A2)=0.05$:

Письмо 1: $x = [1.5 \ 2.5]$

Решение: A1

$P(A1|x) = 0.9521$, $P(A2|x) = 0.0479$

Письмо 2: $x = [2.5 \ 4.]$

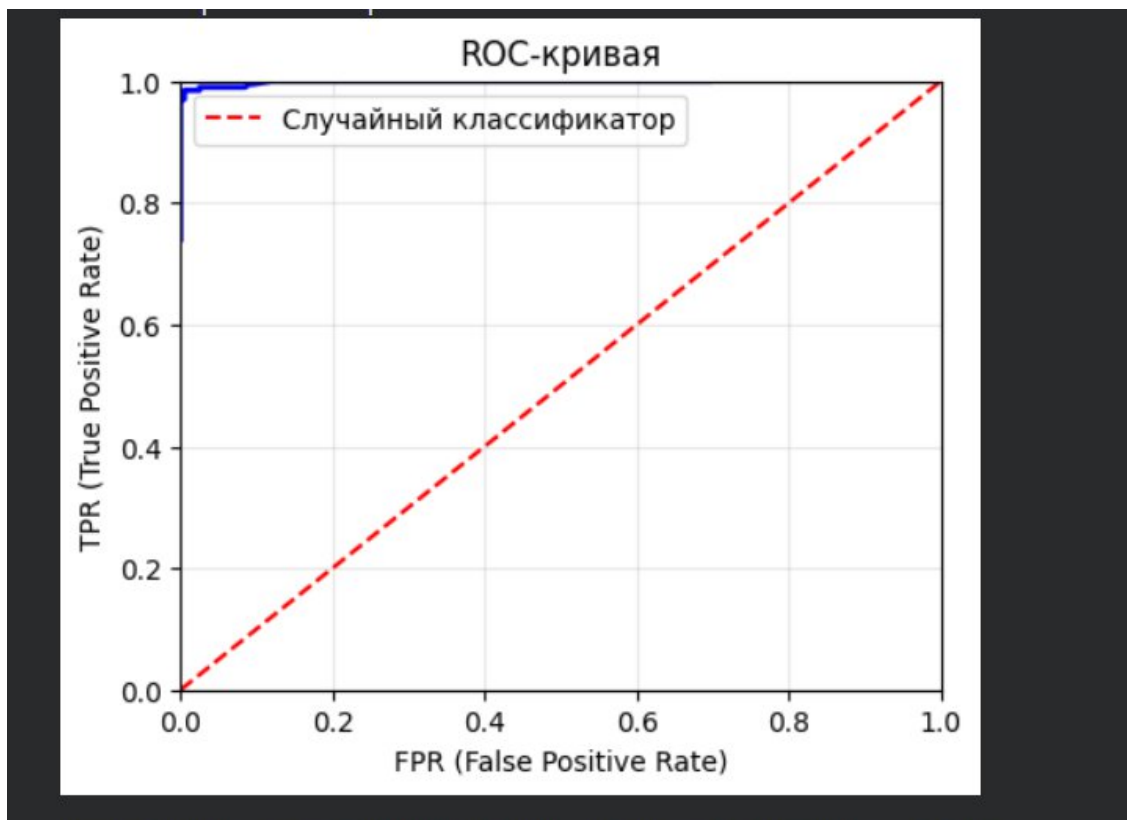
Решение: A2

$P(A1|x) = 0.0002$, $P(A2|x) = 0.9998$

Влияние априорных вероятностей на классификацию:

Тестовая точка: $x = [2. \ 3.]$

$P(A1)$	$P(A2)$	Решение
0.95	0.05	A1
0.80	0.20	A1
0.60	0.40	A2
0.50	0.50	A2



Задание 6. Многомерная классификация

Условие:

Система мониторинга сервера классифицирует состояние по четырем признакам:

- x_1 — загрузка CPU (%)
- x_2 — использование RAM (%)
- x_3 — сетевой трафик (Мб/с)
- x_4 — количество активных процессов

Два класса:

Класс A_1 (нормальная работа):

- $P(A_1) = 0.9$
- $m_1 = [30, 50, 10, 100]^T$
- C_1 = диагональная матрица с элементами $[100, 225, 25, 400]$

Класс A_2 (перегрузка):

- $P(A_2) = 0.1$
- $m_2 = [80, 90, 50, 200]^T$
- C_2 = диагональная матрица с элементами $[225, 100, 100, 900]$

Требуется:

1. Построить 4-мерный классификатор
2. Классифицировать состояния:
 - $x^{(1)} = [50, 70, 25, 150]^T$
 - $x^{(2)} = [75, 85, 45, 180]^T$
3. Вычислить расстояние Махаланобиса
4. Провести анализ важности признаков (feature importance)
5. Построить проекции на двумерные подпространства

Решение:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

class BayesianClassifier:
    def __init__(self):
        self.classes = {}
        self.n_features = None

    def add_class(self, class_name: str, prior: float, mean:
np.ndarray, cov: np.ndarray):
        if self.n_features is None:
            self.n_features = len(mean)

        self.classes[class_name] = {
            'prior': prior,
            'mean': np.array(mean),
            'cov': np.array(cov),
            'cov_inv': np.linalg.inv(cov),
            'cov_det': np.linalg.det(cov)
        }

    def discriminant_function(self, x: np.ndarray, class_name:
str) -> float:
        cls = self.classes[class_name]
        diff = x - cls['mean']
        mahalanobis_sq = diff.T @ cls['cov_inv'] @ diff
        g = np.log(cls['prior']) - 0.5 * np.log(cls['cov_det'])
        - 0.5 * mahalanobis_sq
        return g

    def predict(self, x: np.ndarray):
        scores = {}
        for class_name in self.classes:
            scores[class_name] = self.discriminant_function(x,
```

```

class_name)
    predicted_class = max(scores, key=scores.get)
    return predicted_class, scores

    def mahalanobis_distance(self, class1: str, class2: str,
common_cov: np.ndarray = None) -> float:
    m1 = self.classes[class1]['mean']
    m2 = self.classes[class2]['mean']

    if common_cov is None:
        c1 = self.classes[class1]['cov']
        c2 = self.classes[class2]['cov']
        common_cov = (c1 + c2) / 2

    cov_inv = np.linalg.inv(common_cov)
    diff = m1 - m2
    r_sq = diff.T @ cov_inv @ diff
    return np.sqrt(r_sq)

m1 = np.array([30, 50, 10, 100])
C1 = np.diag([100, 225, 25, 400])

m2 = np.array([80, 90, 50, 200])
C2 = np.diag([225, 100, 100, 900])

classifier = BayesianClassifier()
classifier.add_class('A1', 0.9, m1, C1)
classifier.add_class('A2', 0.1, m2, C2)

test_samples = [np.array([50, 70, 25, 150]), np.array([75, 85,
45, 180])]

print("Классификация 4-мерных состояний сервера:")
for idx, x in enumerate(test_samples, 1):
    pred_class, scores = classifier.predict(x)
    print(f"\nСостояние {idx}: x = {x}")
    print(f"Решение: {pred_class}")
    for name, score in scores.items():
        print(f"    g_{name}(x) = {score:.4f}")

r = classifier.mahalanobis_distance('A1', 'A2')
print(f"\nРасстояние Махаланобиса: r = {r:.4f}")

feature_names = ['CPU', 'RAM', 'Трафик', 'Процессы']
diff = np.abs(m1 - m2)
normalized_diff = diff / np.sqrt(np.diag(C1))

```

```

print("\nАнализ важности признаков:")
importance = normalized_diff / np.sum(normalized_diff)
for i, name in enumerate(feature_names):
    print(f"{name}: {importance[i]:.4f}")

projections = [(0, 1, 'CPU', 'RAM'),
               (0, 2, 'CPU', 'Трафик'),
               (1, 2, 'RAM', 'Трафик')]

fig, axes = plt.subplots(1, 3, figsize=(8, 3))

for ax, (i, j, name_i, name_j) in zip(axes, projections):
    x_range = np.linspace(m1[i]-30, m2[i]+30, 100)
    y_range = np.linspace(min(m1[j], m2[j])-30, max(m1[j],
m2[j])+30, 100)
    X, Y = np.meshgrid(x_range, y_range)
    Z = np.zeros_like(X)

    for row in range(X.shape[0]):
        for col in range(X.shape[1]):
            x_full = np.zeros(4)
            x_full[i] = X[row, col]
            x_full[j] = Y[row, col]
            x_full[[k for k in range(4) if k not in [i, j]]] = \
                [(m1[k] + m2[k])/2 for k in range(4) if k not in
[i, j]]

            pred_class, _ = classifier.predict(x_full)
            Z[row, col] = 0 if pred_class == 'A1' else 1

    ax.contourf(X, Y, Z, levels=[-0.5, 0.5, 1.5],
                colors=['lightblue', 'lightcoral'], alpha=0.5)
    ax.plot(m1[i], m1[j], 'bo', markersize=10, label='A1')
    ax.plot(m2[i], m2[j], 'rs', markersize=10, label='A2')

    for idx, sample in enumerate(test_samples):
        pred, _ = classifier.predict(sample)
        color = 'blue' if pred == 'A1' else 'red'
        ax.plot(sample[i], sample[j], 'X', markersize=8,
color=color)

    ax.set_xlabel(name_i)
    ax.set_ylabel(name_j)
    ax.set_title(f'Проекция на {name_i}-{name_j}')

```

```

ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Вывод:

Классификация 4-мерных состояний сервера:

Состояние 1: $x = [50 \ 70 \ 25 \ 150]$

Решение: A1

$g_{A1}(x) = -20.2351$

$g_{A2}(x) = -21.5309$

Состояние 2: $x = [75 \ 85 \ 45 \ 180]$

Решение: A2

$g_{A1}(x) = -55.0684$

$g_{A2}(x) = -13.5448$

Расстояние Махаланобиса: $r = 8.1373$

Анализ важности признаков:

CPU: 0.2419

RAM: 0.1290

Трафик: 0.3871

Процессы: 0.2419

